

Project Overview

Build an extensible plugin architecture for Gemini CLI to enable custom commands, integrations, and agent workflows.

Problem Statement

Gemini CLI currently lacks a standardized mechanism to extend functionality, making it difficult to integrate custom tools and workflows.

Proposed Solution

Design a modular plugin system that allows developers to create, register, and execute plugins with defined interfaces and configuration support.

Technical Design

Implement a plugin manager, dynamic module loader, configuration-based plugin registration, and sandboxed execution using Node.js capabilities.

Implementation Plan

Phase 1: Analyze codebase and design plugin architecture. Phase 2: Develop plugin manager and loader. Phase 3: Add configuration system and sandboxing. Phase 4: Testing, documentation, and example plugins.

Deliverables

Plugin system integrated into Gemini CLI, sample plugins, and documentation for plugin development.

Challenges & Risks

Handling security, ensuring compatibility with existing CLI structure, and managing plugin execution safely.

Why Me

Strong foundation in Node.js, TypeScript, and building scalable systems with focus on real-world projects.

Future Scope

Support for plugin marketplace, advanced integrations, and community-driven extensions.